IE 4727 Web Application Design

Week 5

Lecturer: Dr. Hu Xiao

JavaScript

- **JavaScript**
  - **Object Creation and Modification**
  - **The Document Object Model (DOM)**
  - **Three ways to access document object in DOM**
  - **Functions**
  - **Event & Event Handler in JS**

• Object creation and modification

Object Creation and Modification

- Objects can be created with the `new` operator followed by a function
- The most basic object is one that uses the `Object()` constructor, as in

```
var myObject = new Object();
```

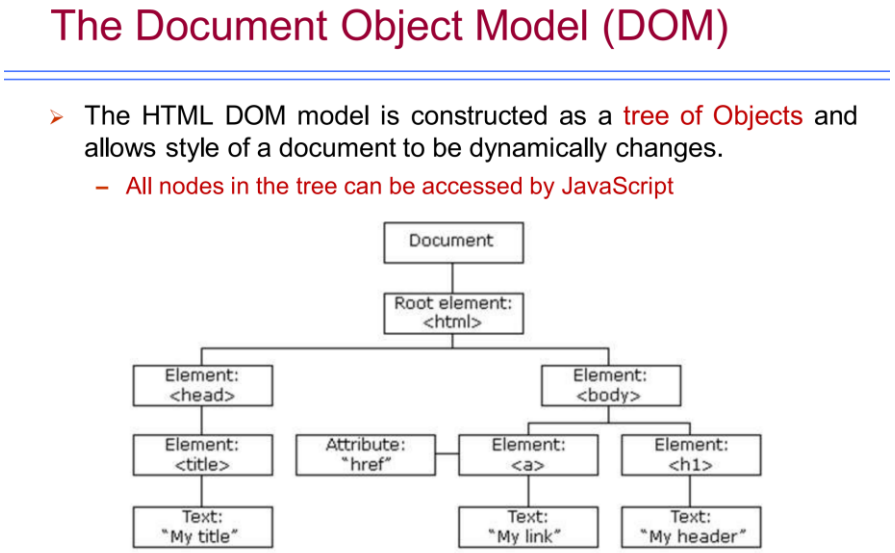
- The new object has no properties - a blank object
- Properties can be added to an object, any time

```
var myAirplane = new Object();
   myAirplane.make = "Cessna";
   myAirplane.model = "Centurian";
```

- Properties can be accessed by dot or in array notations:

```
var property1 = myAirplane["model"];
delete myAirplane.model;
```

• The Document Object Model (DOM)



The Document Object Model (DOM):

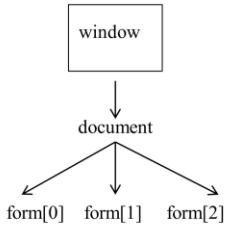
- The DOM is a programming interface for web documents. It represents the page so that programs can manipulate the structure, style, and content of the page., It converts the HTML or XML document into a tree structure where each node is an object representing part of the document.

- The DOM represents the document as a tree of nodes. Each node in the tree corresponds to a part of the document (elements, attributes, text, etc.). All nodes in the tree can be accessed by JS.

• Three ways to access document object in DOM

Accessing Document Objects in DOM

- JavaScript creates an array of all forms, images, links and subordinate objects in a document.
  - Can be accessed by address, name, or method.



1. By address reference

1. Access using address reference

Example (a document with just one form and one widget):

```
<form action = "">
    <input type = "button"  name = "pushMe" />
</form>
```

- Element address

```
document.forms[0].element[0]
```

• Three ways to access document object in DOM

2. By name reference

2. Access using name reference

Same example with `form` named `myForm`

```
<form name = "myForm"  action = "">
    <input type = "button"  name = "pushMe" />
</form>
```

- Element name

```
document.myForm.pushMe
```

3. By DOM Methods

3. Accessing document objects using DOM methods

- getElementById()
  - returns the element with the specified ID
- getElementsByTagName() (array)
  - returns all elements with a specified tag name
- getElementsByClassName (array)
  - returns all elements with the same class name

Example using `getElementById` method

```
<form action = "">
   <input type = "button"  id = "pushMe" />
</form>
```

- DOM object Id referenced

```
document.getElementById("pushMe")
```

- **Functions**

Functions

- A function is a block of code that execute when "someone" calls it
  - block code (inside curly { } braces), preceded by the function keyword
  - Must be declared before they are used and normally in the <head> tag
- Format - *function name begins with a **small** letter*

```
function function_name([formal_parameters]) {
    statements; statements;
}
```

- Parameters can be passed by value or pass-by-reference variables.
- No type checking nor number of parameters checked (excess actual parameters are ignored, excess formal parameters are set to `undefined`).
- All parameters are sent through a property array, `arguments`, which has the `length` property.
  - Return value is the parameter of `return`
  - If there is no `return` at the end of function, `undefined` is returned
  - If `return` has no parameter, `undefined` is returned

• Event & Event Handler in JS

Event: An event is an occurrence that the system or user triggers, which can affect the state or behavior of a program.

Event handler: is a function or method in your code that is specifically designed to respond to a particular event. When an event occurs, the event handler is invoked to process the event and execute the appropriate code.

Event	*Tag Attribute*
blur	onblur
change	onchange
click	onclick
dblclick	ondblclick
focus	onfocus
keydown	onkeydown
keypress	onkeypress
keyup	onkeyup
load	onload
mousedown	onmousedown
mousemove	onmousemove
mouseout	onmouseout
mouseover	onmouseover
mouseup	onmouseup
reset	onreset
select	onselect
submit	onsubmit
unload	onunload

- An event is a notification that something specific has occurred, either with the browser or an action of the browser user
- An event handler is a script that is implicitly executed in response to the occurrence of an event
- The process of connecting an event handler to an event is called registration
- Don't use document.write in an event handler, because the output may go on top of the display

• Sequence of Events

Event Registration: This is the process of defining which events the program should listen for. An event can be anything like a user clicking a button, a keypress, a message being received, or a timer expiring. In this step, the program needs to know what events to watch out for.

Event Listener: Once the events are registered, the program attaches listeners to those events. An event listener is a function or a method that waits for the event to occur. When the event happens, the listener triggers the execution of the associated code. This is where you specify what should happen when a particular event occurs.

Event Handling: This is the final step, where the actual handling of the event takes place. When the event listener detects an event, it triggers the event handler—a function or block of code that defines what actions to perform in response to the event. For example, if a user clicks a button, the event handler might update the user interface or submit data to a server.

Week 05 Lab Exercise

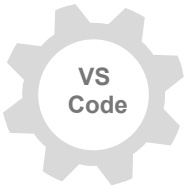
1. **Complete case study part 03 Create first React App. (optional). Find requirements for CS 03 on NTULearn main course site under Content-> Case Studies -> Case Study 03. Please watch the briefing video Case Study 03_video before implementation.**

2. **Prepare for Quiz 1, which will be held during the first 15 minutes of the lab session in Week 6, Quiz 1 will test you the theoretical understanding of important concepts on the HTML5, CSS3, and JavaScript. Quiz 1 will be in MCQ and short-answer format, contents will cover up to Lecture 6 JavaScript (The first 6 weeks lecture topics).**

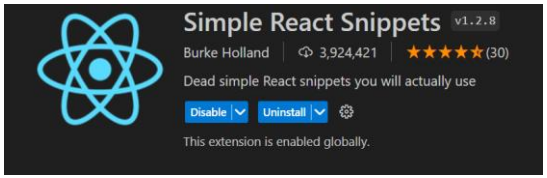
Case Study 03 (optional)

1. Download Node.js from https://nodejs.org/en/download and add Node.js to PC environment variables

- React apps requires Node.js to run.
- Node.js includes npm (Node Package Manager) used for managing dependencies and scripts
- Verify installation using terminal/command line (node –v and npm -v).



2. Download and Install React Extension in VS Code



- Open VS Code. Install the **Simple React Snippets** extension to help with React development.

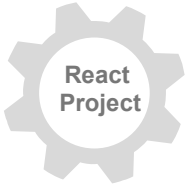
3. Install vite: Within the VS Code terminal, enter **npm create vite@latest**, and hit enter.

- Vite is a fast build tool for modern web projects (React, Vue, etc).
  - In this couse, we use Vite+ReactJS to create React projects

4. Create new React project in VS Code: **npm create vite@latest <project name> -- -- template react**

```
PS C:\Users\65852\Desktop\FirstReact> npm create vite@latest firstReactApp -- --template react
√ Package name: ... firstreactapp

Scaffolding project in C:\Users\65852\Desktop\FirstReact\firstReactApp...

Done. Now run:

  cd firstReactApp
  npm install
  npm run dev
```

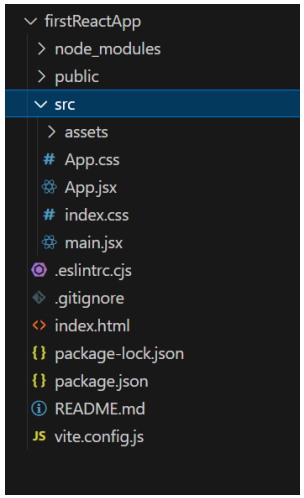
Download and run React App



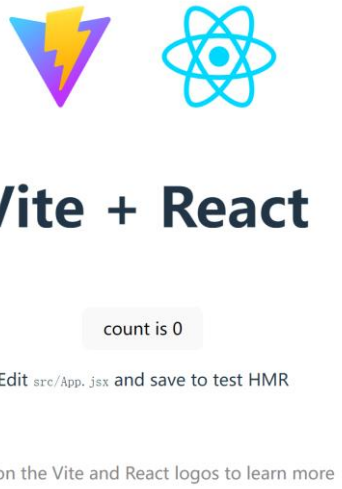
4. Create new React project in VS Code: **npm create vite@latest <project name> -- -- template react**

```
PS C:\Users\65852\Desktop\FirstReact> npm create vite@latest firstReactApp -- --template react
√ Package name: ... firstreactapp

Scaffolding project in C:\Users\65852\Desktop\FirstReact\firstReactApp...

Done. Now run:

  cd firstReactApp
  npm install
  npm run dev
```

5. Type **cd firstReactApp** -> **npm i** -> **npm run dev**




```
VITE v5.2.10  ready in 454 ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
➜  press h + enter to show help
```



Case Study 03: Form Validation Using React.js

Targets: 1. Build the login form in ReactJS and use the useState hook to manage form data.
2. Implement basic form validation (valid email format, non-empty input, and etc.

01 FormValidationExample.jsx

Create form contents in .jsx file

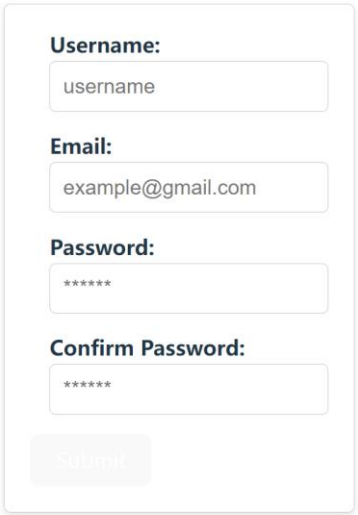
```jsx
import { useState } from 'react'

import './App.css'


const FormValidationExample = () => {

  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: '',
    confirmPassword: ''
  })
```

App.css

Add sheet styles (.css file) on the form

```css

/* Styles for the form container */
* {
  box-sizing: border-box;
  margin-left: auto;margin-right: auto;

}
form {
  max-width: 400px;
  margin-left: 0px;
```

02 Vite+React on Browser



Thanks

Lecturer: Dr. Hu Xiao

Email: xiao.hu@ntu.edu.sg